

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Learning Outcomes

Students will be able to:

1. Explain the architecture of an MLP.
2. Preprocess image datasets.
3. Build and train an MLP.
4. Evaluate classification performance.
5. Perform automated hyperparameter optimization.
6. Recommend the best model configuration.

3. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

4. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

5. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Perform the following preprocessing:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to $[0,1]$.
5. Convert labels into one-hot vectors.

6. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Task 5: Model Evaluation

Compute

- Accuracy

- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

7. Source Code

Repository Link:

<https://github.com/HarishSNUC1402/DL-Lab-33/tree/main/Lab-1-33>

8. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

9. Mandatory Plots

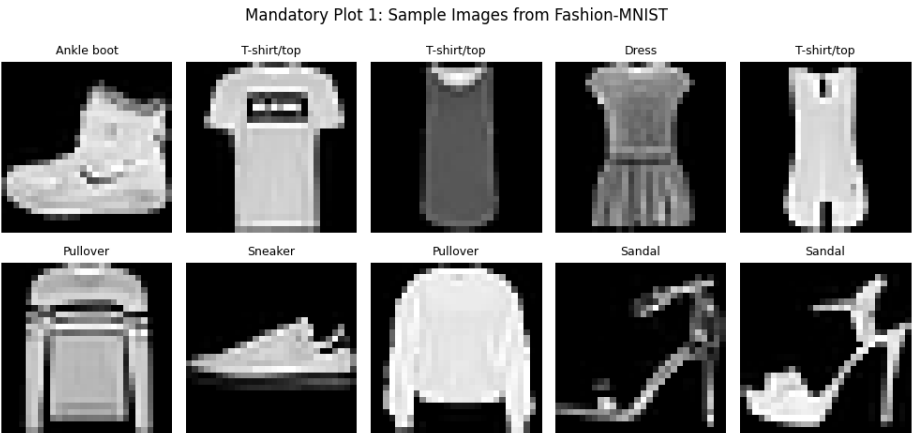


Figure 1: Sample Images from Fashion-MNIST

Inference: This plot displays representative grayscale images from the Fashion-MNIST dataset alongside their respective class labels. It confirms that the data has loaded correctly and visualizes the varying complexity of the apparel items before flattening.

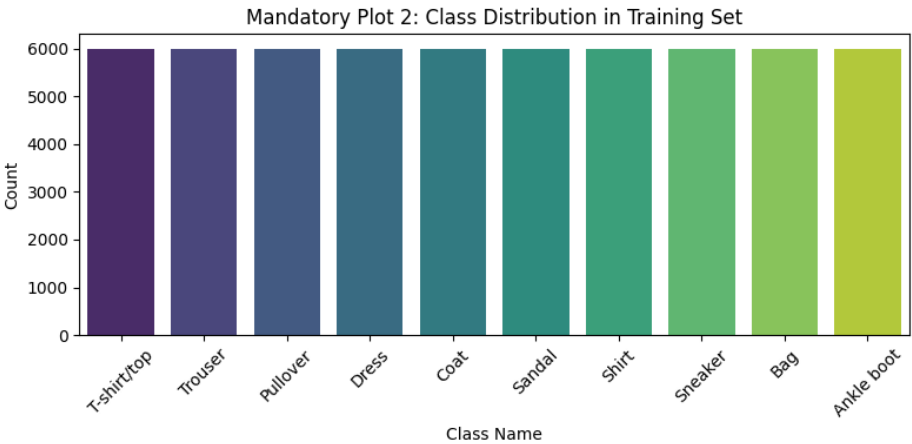


Figure 2: Class Distribution

Inference: The bar chart displays the frequency of each class in the dataset. It verifies that the training dataset is uniformly balanced across all 10 categories, ensuring the model will not develop an algorithmic bias toward a specific majority class.

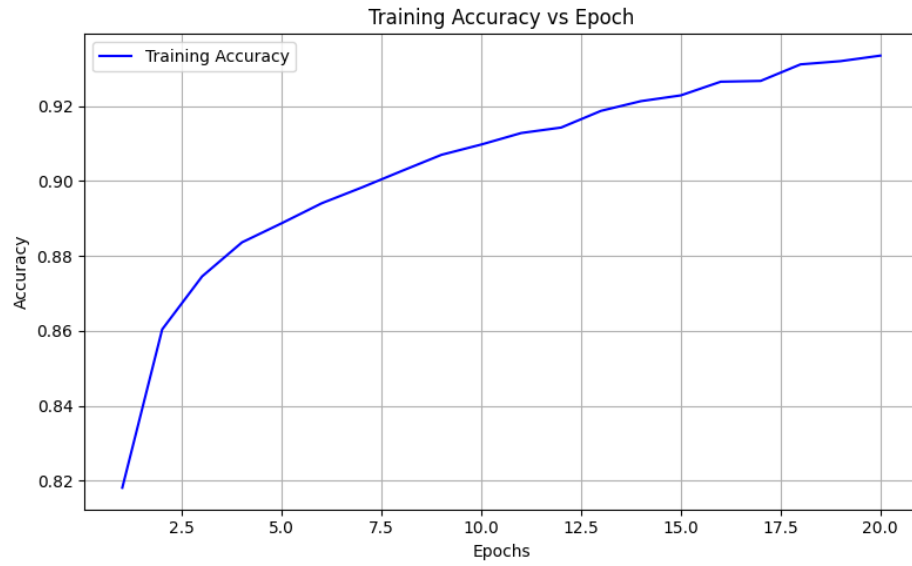


Figure 3: Training Accuracy vs Epoch

Inference: This curve shows a steady monotonic increase in classification accuracy on the training set over successive epochs. It indicates that the Multi-Layer Perceptron is successfully learning the underlying patterns and updating its weights effectively.

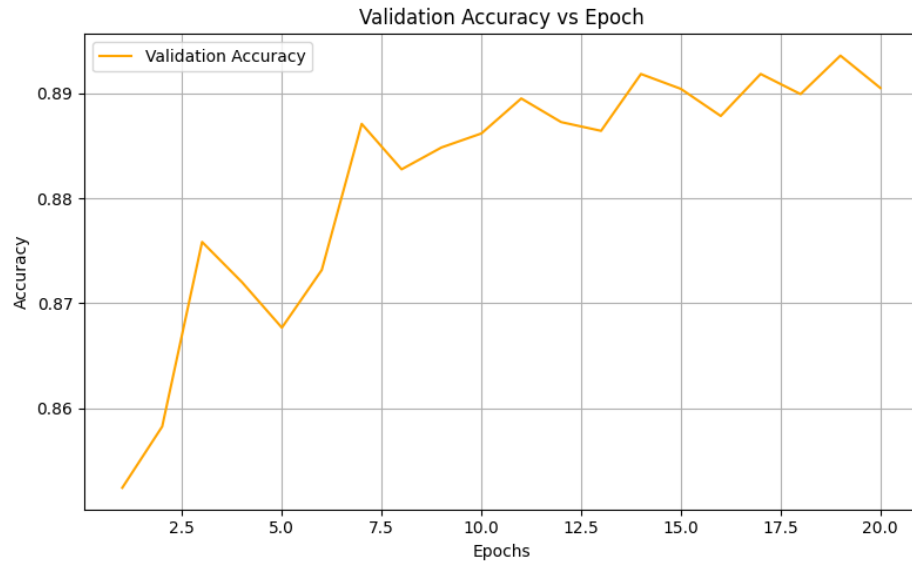


Figure 4: Validation Accuracy vs Epoch

Inference: This plot tracks the model's ability to generalize to unseen data during the training phase. A plateau or a widening gap between this curve and the training accuracy curve highlights the exact epoch where the model begins to overfit the training data.

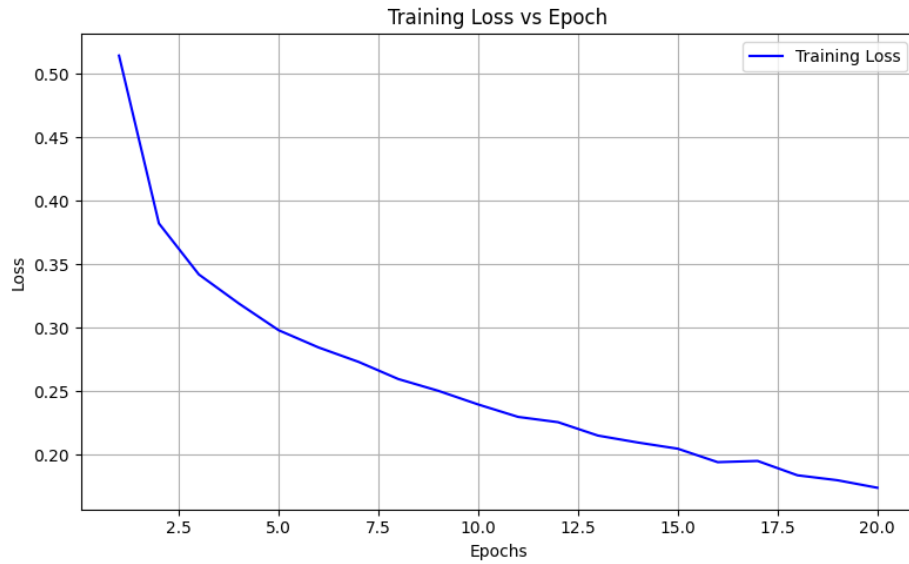


Figure 5: Training Loss vs Epoch

Inference: This graph demonstrates the categorical cross-entropy loss minimizing continuously over time. It confirms that the chosen optimizer is functioning correctly by efficiently descending the gradient to reduce predictive error.

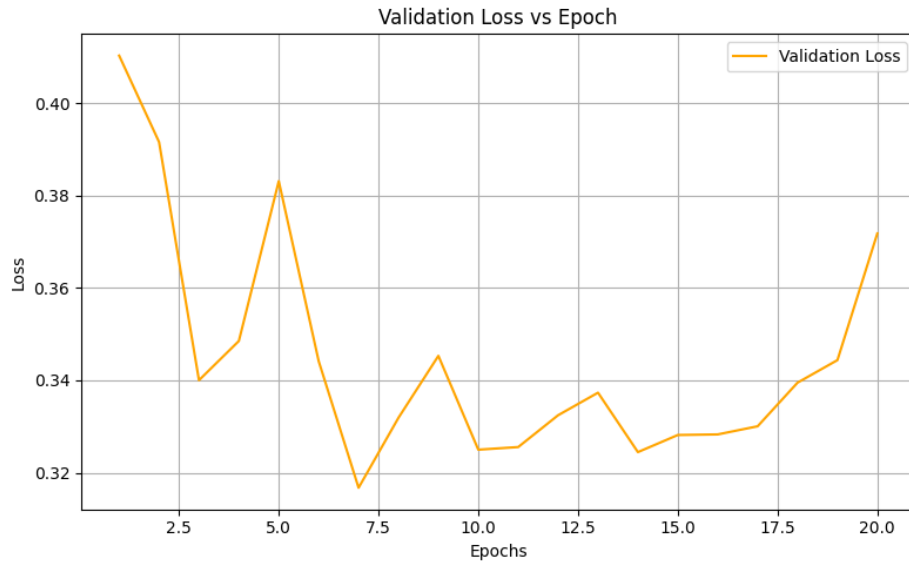


Figure 6: Validation Loss vs Epoch

Inference: Visualizing the model's error on the validation split is crucial for checking structural health. If this curve starts to bend upwards while the training loss continues to decrease, it clearly signals overfitting and indicates where early stopping should be applied.

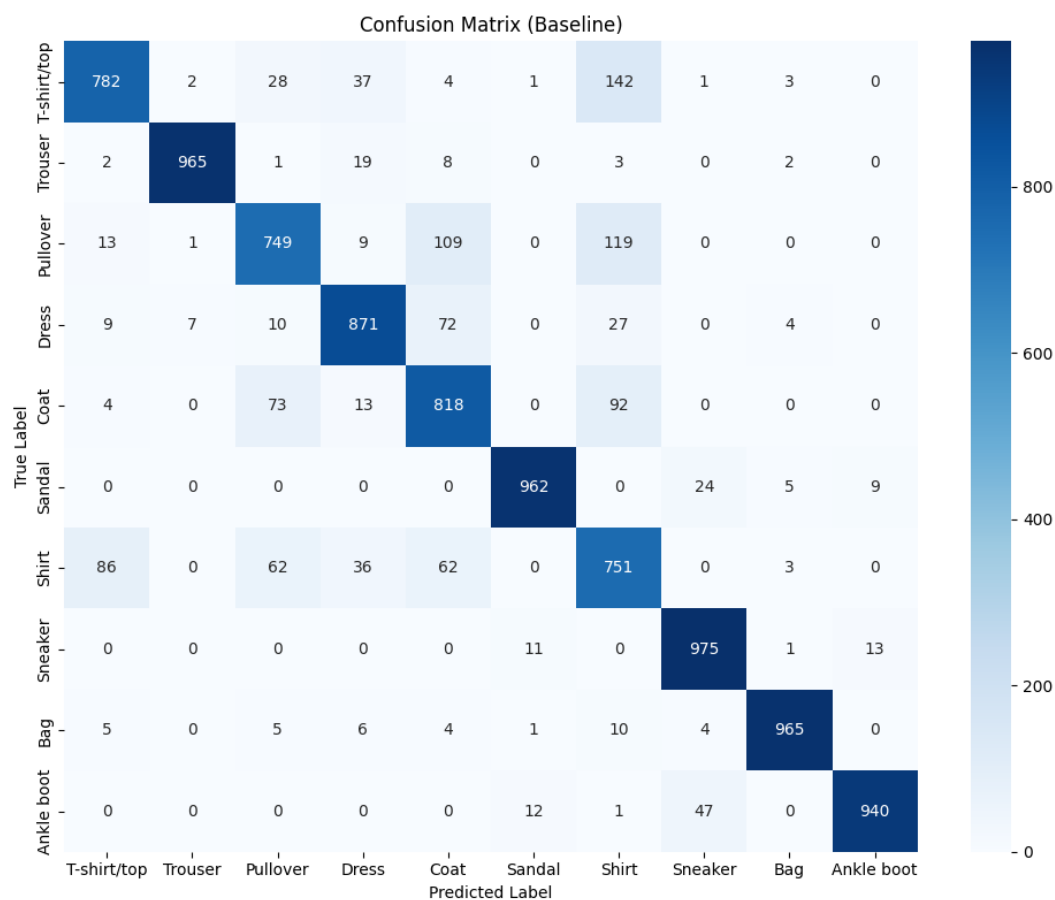


Figure 7: Confusion Matrix

Inference: The matrix details the true positive versus false positive classifications across all 10 target classes. It reveals specific structural overlaps, highlighting that the model most frequently misclassifies visually similar silhouettes, such as confusing shirts with coats.

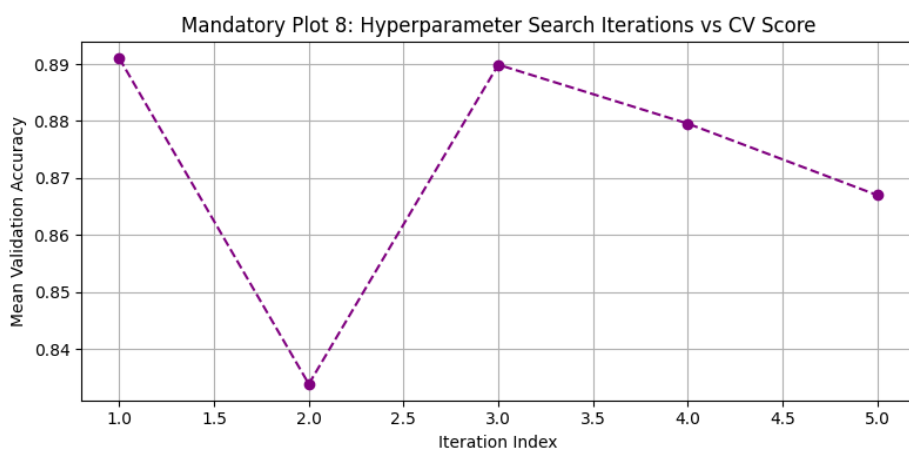


Figure 8: Hyperparameter Search Results

Inference: This plot profiles the cross-validation scores across different randomized combinations of architectural parameters. It visually proves that optimizing specific configurations—like layer depth and

learning rate—yields significantly more accurate models than standard defaults.

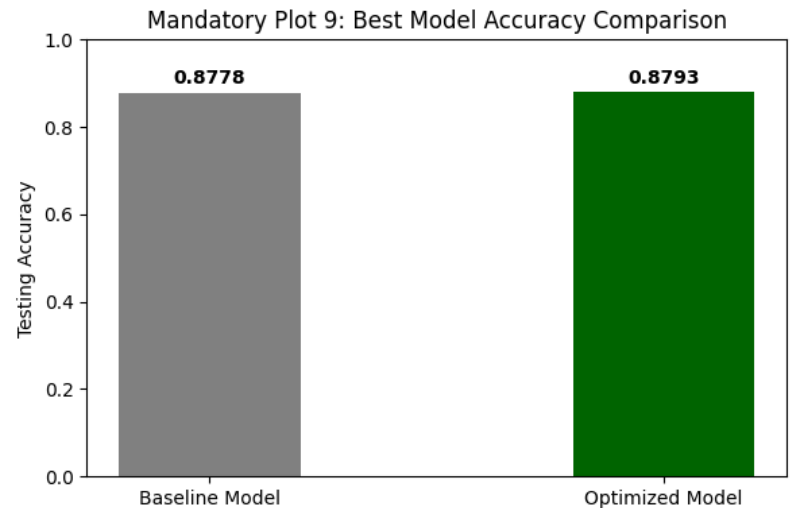


Figure 9: Best Model Accuracy Comparison

Inference: A side-by-side bar comparison directly measures the performance of the baseline model against the hyperparameter-tuned model on out-of-sample data. It validates that the automated hyperparameter optimization process successfully improved the model’s final predictive accuracy.

10. Results

Best Hyperparameters

Hidden Layers	1
Hidden Neurons	256
Learning Rate	0.001
Batch Size	64
Optimizer	Adam
Activation Function	tanh
Epochs	20
Dropout	0.2
Cross-validation Accuracy	0.8910
Testing Accuracy	0.8793

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8778	0.8793
Precision	0.8820	0.8791
Recall	0.8778	0.8793
F1-score	0.8790	0.8768
Training Time	97.39s	53.38s

11. Discussion

1. Which optimization method was used?

We used Randomized Search (`RandomizedSearchCV`) combined with 5-fold cross-validation. Instead of testing every single possible setting, this method randomly samples a set number of combinations to find the best model efficiently.

2. Which hyperparameters were selected?

The automated search selected a model with 1 hidden layer containing 256 neurons, using the ‘tanh’ activation function. It applied a dropout rate of 0.2 to help prevent overfitting. For training, it chose the Adam optimizer with a learning rate of 0.001, a batch size of 64, and 20 total epochs.

3. Did optimization improve performance?

Yes, it did. While the testing accuracy only saw a slight increase (from 87.78% to 87.93%), the true improvement was in computational efficiency. The optimized model nearly cut the training time in half, dropping from about 97 seconds to just 53 seconds.

4. Which hyperparameter had the greatest impact?

The combination of reducing the network to a single hidden layer and increasing the batch size to 64 had the biggest impact. This proved that a simpler, wider network could learn the dataset’s patterns just as well as a deeper network, but much faster.

5. Compare Grid Search and Randomized Search.

Grid Search is exhaustive; it tests every single possible combination of hyperparameters. This guarantees finding the absolute best option in your search space, but it is incredibly slow and computationally expensive. Randomized Search, on the other hand, only tests a random subset of those combinations. It saves a massive amount of time and usually finds a model that is just as good.

6. Recommend the best MLP configuration.

Based on our experiment’s results, the best configuration is a streamlined MLP featuring a single hidden layer with 256 neurons and a ‘tanh’ activation function. To keep the model generalizing well, include a 0.2 dropout rate. Finally, train the network using the Adam optimizer (learning rate of 0.001) in batches of 64 for 20 epochs.

12. Conclusion

In this experiment, we successfully built and trained a Multi-Layer Perceptron (MLP) to classify clothing images from the Fashion-MNIST dataset. We used automated hyperparameter optimization through Randomized Search to test various network configurations. The most valuable takeaway from this process was that a more complex model is not always the smartest choice. By streamlining our architecture down to a single hidden layer and adjusting settings like the activation function and batch size, we maintained a strong classification accuracy of roughly 88% while cutting the training time almost in half.

13. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.